

大模型及其驱动的编程工具 ——科研与开发的实用指南

2026 年 1 月更新版

许达

未来院三室

2026 年 1 月 8 日

内容大纲

第一部分：大模型基础与评测

- 大模型发展概览
- 科研能力评测 (HLE/FrontierMath)
- 软件开发评测 (SWE-bench)
- AI vs 人类程序员
- 技术原理 (MoE/推理时计算)
- 主流 AI 大模型详解
- 大模型的核心问题
- **Lost in the Middle 现象**

第二部分：模型深入解析

- 校准误差与可信度
- DeepSeek/Gemini 详解

第三部分：AI 编程工具

- **Agent 工作流简介**
- AI 编程工具实战

第四部分：科研应用实践

- 科研场景应用
- 实用技巧与最佳实践

第五部分：高级技巧

- 高级技巧与前沿话题
- API 省钱与数据安全

第六部分：风险与规范

- AI 论文检测与学术诚信
- **科研 LLM 应用风险**

第一部分：大模型基础与评测

三大主流厂商:

-  OpenAI - GPT-5.2 系列
-  Anthropic - Claude Opus 4.5
-  Google - Gemini 3 Pro

中国优秀模型:

-  DeepSeek - V3.2 (开源最强)
- 阿里云 - Qwen 3.0
- 月之暗面 - Kimi K2

重要提示

不同任务适合不同模型，详细评测数据和模型选择建议见后续章节。

模型架构解析：MoE（混合专家） vs Dense（密集）

什么是 MoE?

Mixture of Experts（混合专家）

- 模型包含多个“专家”子网络
- 每次推理只激活**部分专家**
- 路由器决定使用哪些专家
- 总参数量大，但计算量小

MoE 优势

- **✓** 训练效率高（同算力训更大模型）
- **✓** 推理计算少（只用部分参数）
- **✓** 成本更低

MoE 劣势

许达（未来院三室）

主流模型架构对比

模型	架构	MoE 规格
— MoE 架构 —		
DeepSeek V3.2	MoE	671B/37B 激活
Gemini 3 系列	MoE	未公开
Mixtral 8x7B	MoE	47B/12B 激活
Grok 4	MoE	推测
Qwen3-MoE	MoE	多版本
— Dense 架构 —		
GPT-5.2	Dense	—
Claude 4.5	Dense	—
Llama 4	Dense	—

DeepSeek MoE 详解

- **总参数**：671B（6710 亿）
- **激活参数**：仅 37B（5.5%）

推理时计算扩展：成本倒挂现象

两种扩展范式

传统：训练时扩展

- 更大模型 + 更多数据
- GPT-3 → GPT-4 → GPT-5
- 训练成本极高，推理成本固定

新范式：推理时扩展

- 让模型”思考更久”
- 复杂问题多花时间推理
- 训练成本降低，推理按需付费

推理成本倒挂：厂商在亏钱！

OpenAI 的困境 (2024-2025 数据):

- 推理部署成本：每次查询 \$0.01-0.10
- 实际收费：远低于成本
- 2024 年预计亏损：\$50 亿
- 日运营成本：约 \$70 万/天

为什么还要做？

- 抢占市场份额优先于盈利
- 期望规模效应降低成本
- 依赖投资支撑 (\$170 亿融资)

模型	类型	推理特性	适用场景	思考控制
GPT-5.2	传统	快速响应	通用对话、写作	无
GPT-5.2 (推理模式)	推理型	内置深度思考	数学、逻辑、编程难题	自动

ChatGPT 订阅价格详解 (2026 年 1 月)

套餐	价格	模型	主要功能
Free	免费	GPT-5.2-mini	基础对话、有限次数 GPT-5.2、联网搜索、代码执行
Plus	\$20/月	GPT-5.2 全系列	更多 GPT-5.2 额度、图像生成 (DALL·E)、语音对话、自定义 GPTs
Pro	\$200/月	全部模型	无限 GPT-5.2 访问、最高推理能力、研究级任务
Team	\$25-30/人/月	GPT-5.2 全系列	团队协作、更长上下文、管理控制台、数据不训练
Enterprise	联系销售	全部	无限高速、企业 SSO、高级安全、专属支持、SOC2 合规

API 定价 (按 Token 计费)

推荐选择

● 轻度用户 → Free 足够

模型家族 (2026 年 1 月最新):

- **Claude Sonnet 4.5** (9/30 发布)
世界最强编程模型 (SWE-bench 77.2%)
OSWorld 61.4% (计算机使用 SOTA)
- **Claude Opus 4.5** (11/25 发布)
编程、Agent、Computer Use 全能旗舰
- **Claude Haiku 4.5** (10/16 发布)
超快速度, 极具性价比

Claude 订阅价格

- **Pro**: \$20/月
- **Max 5x**: \$100/月
- **Max 20x**: \$200/月
- **Team**: \$30/用户/月
- **Enterprise**: 定制价格

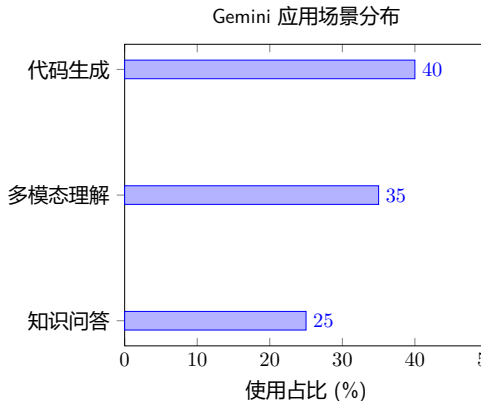
估值飙升

9 月: Series F 融资 \$130 亿
估值: \$1830 亿
收入: 8 个月从 \$10 亿 → \$50 亿 +

模型家族 (2026 年 1 月最新):

- Gemini 3 Pro (HLE 37.5%, 科研第一)
- Gemini 3 Flash (12 月——极速前沿智能)
- Gemini 3 Deep Think (推理增强版)
- Nano Banana Pro (轻量版图像生成)

适用场景: 科研分析、多模态理解、Google Workspace 集成



最新版本 (2026 年 1 月全球媒体报道):

- **DeepSeek-V3.2 正式版** (1 月发布)
强化 Agent 能力, 融入思考推理
- **DeepSeek-V3** (12/26 发布)
671B MoE 参数, 60 tokens/s (3 倍 V2 速度)

核心特点:

- **中文能力全球第一**：在中文语境理解、古文、成语及中国文化相关任务上碾压所有国外模型
- **完全免费**网页端和 APP 使用
- **开源**：671B MoE + 37B 激活参数
- **API 价格全球最低**！

适用场景：

- ### ● 所有中文相关任务的首选

国产之光

DeepSeek 不仅是性价比之选，更是**中文领域的最强王者**。对于国内科研人员和开发者，它是处理中文文档和数据的最佳工具。

访问方式:

<https://chat.deepseek.com>

<https://platform.deepseek.com>

大模型对比总结

特性	GPT-5.2 系列	Claude 4.5	Gemini 3 Pro	DeepSeek V3.2
代码能力	极强	最强	强	强
推理能力	极强	极强	极强	极强
多模态	全面	文本为主	最强	基础
上下文长度	128K	200K	2M	128K
中文支持	良好	良好	良好	无敌
价格	较高	中等	中等	最低
免费额度	有限	有限	较多	充足

选择建议

- **复杂编程任务**：Claude Opus 4.5 或 GPT-5.2
- **中文任务/预算敏感**：DeepSeek V3.2 (中文最强)
- **多模态需求**：Gemini 3 Pro 或 GPT-5.2
- **长文本处理**：Gemini 3 Pro (2M) 或 Claude (200K)

著名大模型性能特点详解（一）

GPT-5.2 (OpenAI)

核心优势:

- 综合能力最均衡的“全能选手”
- 生态最成熟，插件/工具最丰富
- 多模态能力强（图像、语音、视频）
- 指令遵循能力极佳

相对弱点:

- 价格较高（API \$2.5-10/M tokens）
- 中文表达略显“翻译腔”
- 知识截止日期更新较慢

最佳场景: 通用问答、创意写作、多模态任务

Claude 4.5 (Anthropic)

核心优势:

- **代码能力公认最强**（SWE-bench 77%）
- 200K 超长上下文，支持整个代码库
- 输出格式规范、逻辑清晰
- Computer Use 功能（操作电脑）

相对弱点:

- 多模态能力不如 GPT/Gemini
- 有时“过于谨慎”，拒绝回答
- 创意写作风格较为正式

最佳场景: 编程开发、代码审查、长文档分析

著名大模型性能特点详解 (二)

Gemini 3 Pro (Google)

核心优势:

- **HLE 科研能力第一** (37.5%)
- **2M 超长上下文** (业界最长)
- 多模态理解最强 (原生多模态)
- Google 服务深度集成

相对弱点:

- 国内访问受限
- 中文表达不如本土模型自然
- 部分编程语言支持较弱

最佳场景: 科研问答、长文档分析、多模态研究

DeepSeek V3.2 (中国)

核心优势:

- **中文能力无敌** (语义、文化理解)
- **价格全球最低** (\$0.27/M 输入)
- 完全免费网页端使用
- 开源, 可本地部署
- 数学推理能力极强

相对弱点:

- 多模态能力较基础
- 英文创意写作略逊
- 生态工具不如 OpenAI 丰富

最佳场景: 中文任务、数学推理、性价比开发

著名大模型性能特点详解（三）

2026 年顶级模型对比

模型	特点
GPT-5.2 (OpenAI)	全能旗舰、生态最强
Gemini 3 Pro	科学推理 SOTA、2M 上下文
Claude Opus 4.5	代码之王、Agent 最强
DeepSeek V3.2	中文最强、性价比王

2026 年模型选择要点：

- 旧版推理模型已被整合到 GPT-5.2
- 新旗舰模型内置推理能力
- 无需单独选择“推理模型”

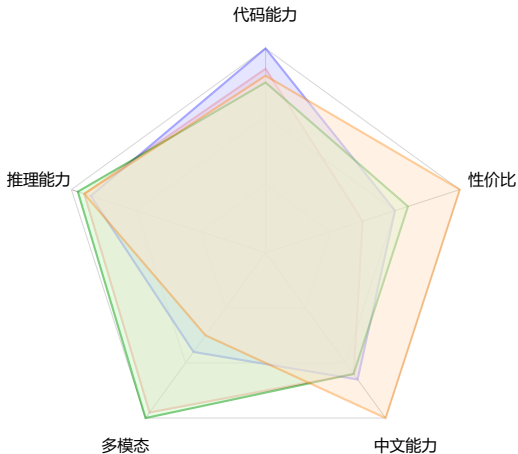
各模型”性格”总结

- GPT-5.2：老成稳重的**全能助手**
- Claude Opus 4.5：严谨专业的**代码专家**
- Gemini 3 Pro：博学多才的**科研伙伴**
- DeepSeek V3.2：懂中文的**性价比之王**

实用建议

- 日常问答：DeepSeek（免费）
- 写代码：Claude Opus 4.5（最强）
- 处理图片/视频：GPT-5.2 或 Gemini 3
- 中文写作：DeepSeek（最自然）
- 科学研究：Gemini 3 Pro

大模型能力雷达图对比



图例说明:

- GPT-5.2
- Claude 4.5
- Gemini 3 Pro
- DeepSeek V3.2

核心洞察

- 没有“最好”的模型，只有“最适合”
- 各模型优势领域明显不同
- 组合使用效果最佳
- 关注任务需求而非品牌

评分基于公开基准测试与实际使用体验

大模型的三大核心问题

⚠ 幻觉 (Hallucination)

一本正经地胡说八道

- 编造不存在的引用
- 虚构历史事件
- 捏造统计数据
- 错误的技术细节

📄 上下文遗忘

聊着聊着就忘了

- 忘记早期指令
- 丢失关键约束
- 前后矛盾
- 重复已说内容

📉 质量衰减

越聊越差

- 多轮后回答变短
- 创意能力下降
- 出现循环重复
- 指令遵循变差

核心认识

这些问题**不是** bug，而是当前技术架构的本质限制。理解原因才能更好地规避。

问题根源（一）：幻觉的本质——“鹦鹉学舌”而非“理解”

大模型的工作原理：

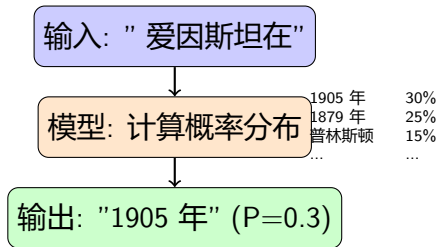
- ① 输入文本 → 转为 Token 序列
- ② 根据上文预测**下一个最可能的 Token**
- ③ 采样生成 → 重复直到结束

关键洞察：

- 模型在做**统计模式匹配**
- **不是在**“理解”或“推理”
- **不是在**查询知识库
- 只是预测“什么词最可能出现在这里”

为什么会幻觉？

- 训练数据中某些模式频繁出现
- 模型学会了“看起来正确”的表达式



本质问题

模型选择“统计上最可能”的续写，而非“事实上正确”的答案。当训练数据不足或有噪声时，就会产生幻觉。

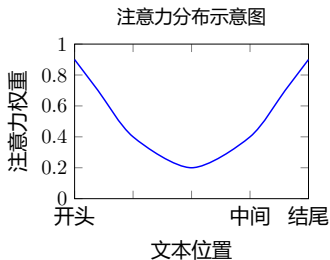
问题根源（二）：上下文遗忘——有限的“工作记忆”

Transformer 的注意力机制：

- 每个 Token 都要关注所有其他 Token
- 计算复杂度： $O(n^2)$ (n 为序列长度)
- **上下文窗口**是硬性限制

上下文窗口限制（2026 年）：

- GPT-5.2: **128K** tokens
- Claude 4.5: **200K** tokens
- Gemini 3: **2M** tokens



“Lost in the Middle” 现象

研究表明，当重要信息放在长文本中间时，模型检索准确率**下降 40-60%**。

Liu et al., 2024, "Lost in the Middle"

关键问题：

- 窗口内的信息**不是均匀关注**
- 存在“Lost in the Middle”现象
- 开头和结尾被关注更多

问题根源（三）：多轮对话质量衰减

多轮对话的技术实现：

- ① 每轮对话都把**完整历史**发给模型
- ② 历史越长，Token 消耗越多
- ③ 模型每次都要重新“阅读”全部内容

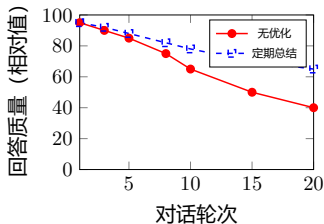
质量衰减的原因：

- **上下文污染**：历史中的错误会被“学习”
- **注意力稀释**：关注点分散到更多内容
- **模式锁定**：倾向重复已有的表达模式
- **指令遗忘**：原始指令权重降低

类比理解

像一个人同时听 10 个人说话，每多一个人，对每个人的注意力就少一点。

多轮对话质量变化



技术解释

为什么会重复？

当历史中某个模式出现多次，模型预测“下一个词”时，这个模式的概率被不断强化，形成“自我循环”。

深层原因：为什么这些问题难以根治？

1. 训练目标的局限

- 目标：预测下一个 Token
- **不是**：生成正确的事实
- **不是**：保持逻辑一致性
- **不是**：长期记忆管理

2. 知识存储方式

- 知识编码在**模型参数**中
- 不是存储在可查询的数据库
- 无法精确检索和验证
- 更新知识需要重新训练

3. 缺乏真正的“理解”

- 没有世界模型

Stochastic Parrot (随机鹦鹉)

Bender et al. (2021) 的著名论文指出：

“大语言模型是**随机鹦鹉**——它们在操纵语言形式，而不理解语言的**意义**。”

模型学会了“说人话”的统计模式，但不理解话语背后的世界。

类比

传统数据库：图书馆，可以精确查找

大语言模型：读过很多书的人，凭印象回答，可能记错细节

实用应对策略

应对幻觉：

① 要求引用来源

- “请给出参考文献”
- 然后人工验证

② 使用 RAG

- 让模型检索真实文档
- 基于文档回答

③ 交叉验证

- 用多个模型验证
- 关键信息人工核实

应对上下文遗忘：

① 重要指令放在**开头和结尾**

② 定期**重复关键约束**

③ 使用**结构化格式** (JSON/XML)

应对质量衰减：

① 定期总结

- “请总结我们讨论的要点”
- 开新对话，粘贴总结继续

② 分段处理

- 复杂任务拆分为多个短对话
- 每个对话专注单一目标

③ 重置指令

- 对话变差时重申原始要求
- “请重新阅读我的初始指令”

黄金法则

AI 是助手，不是权威
永远保持批判性思维，关键信息必须验证

前沿研究：这些问题正在被解决吗？

缓解幻觉的研究方向：

- **RAG (检索增强生成)**
 - 先检索相关文档
 - 基于文档生成答案
 - 显著减少事实错误
- **Self-Consistency**
 - 多次采样取共识
 - 提高答案可靠性
- **知识编辑**
 - 精确修改模型中的错误知识
 - 仍在研究阶段

扩展上下文的研究：

- **稀疏注意力**
 - 不关注所有 Token
 - 降低计算复杂度
- **记忆增强**
 - 外部记忆模块
 - 长期知识存储
- **上下文压缩**
 - 自动总结历史
 - 保留关键信息

现实预期

这些问题在**短期内不会完全解决**，但会持续改善。
学会与这些限制共处是使用 AI 的必要技能。

“Lost in the Middle” 现象详解

什么是“Lost in the Middle”?

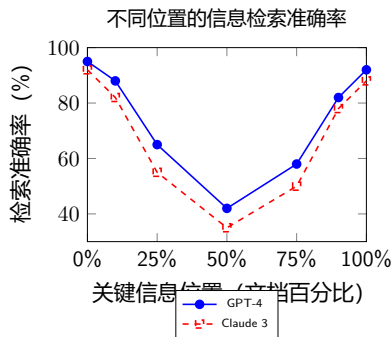
- 大模型处理长文本时的**注意力分布不均**现象
- 开头和结尾的信息被充分关注
- 中间部分的信息容易被“遗忘”**

实验数据 (Liu et al., 2024):

- 测试任务: 长文档问答
- 关键信息在**开头**: 准确率 90%+
- 关键信息在**中间**: 准确率 40-50%
- 关键信息在**结尾**: 准确率 85%+

影响范围:

- 所有 Transformer 架构模型都有此问题



U 型曲线

注意力呈现典型的 **U 型分布**——首尾高、中间低。这是 Transformer 位置编码和注意力机制的固有特性。

应对“Lost in the Middle” 的实用技巧

✓ 文档组织策略:

① 关键信息前置

- 最重要的内容放在开头
- 次重要的放在结尾
- 背景信息放在中间

② 分块处理

- 长文档拆分为多个短段
- 每段独立提问
- 最后综合答案

③ 重复关键信息

- 在开头和结尾重复强调
- “请特别注意以下要求...”

</> Prompt 工程技巧:

示例：长文档问答

【重要指令 - 开头】

请在以下文档中找到关于 X 的信息。

[文档内容...]

【重要提醒 - 结尾】

请确保回答基于文档内容，
特别关注关于 X 的部分。

💡 RAG 最佳实践:

- 检索结果按相关性**重排序**
- 最相关的放在**开头或结尾**
- 每次检索控制在 **5-10 个 chunk**
- 使用**摘要**代替完整文档

第二部分：模型深入解析

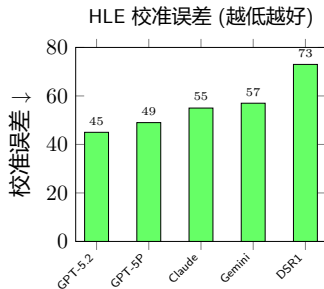
什么是校准误差？——模型自知之明的衡量

校准误差 (Calibration Error) 定义

- 衡量模型**置信度**与**实际准确率**的匹配程度
- 公式: $CE = |\text{模型置信度} - \text{实际正确率}| \times 100$
- **数值越低**, 模型越“知道自己知不知道”

举例说明:

- 模型说“我 90% 确定答案是 A”
- 如果 100 次这样的情况中真的有 90 次正确
→ **校准误差 = 0 (完美校准)**
- 如果实际只有 60 次正确
→ **校准误差 = 30 (过度自信)**



科研应用意义

- 低校准误差 = 可信的不确定性估计
- GPT-5 系列校准最好 (45-49)
- DeepSeek-R1 过度自信 (73)

科研建议: 需要可靠置信度时选 GPT-5 系列; 追求准确率选 Gemini 3 Pro

DeepSeek V3.2: 开源科研利器详解

DeepSeek V3.2/R1 关键数据 (HLE 2025.12)

- **HLE 科研能力**: 8.5% (开源最强)
- **SWE-bench**: 60.0% (仅 \$0.03/任务)
- **架构**: MoE, 671B 参数, 37B 激活
- **特点**:
 - 强化 Agent 能力
 - 融入思考推理
 - 中文理解无敌

开源蒸馏模型 (可本地部署):

- DeepSeek-V3-32B: 适合消费级显卡
- DeepSeek-Coder-V3: 代码专精
- 支持 Ollama 一键部署

科研应用优势

- ✓ 完全开源, MIT 许可
- ✓ 可本地部署
- ✓ 中文推理能力极强
- ✓ API 成本极低 (Claude 的 1/24)
- ✓ 性价比之王

推荐配置

日常使用: DeepSeek V3.2 网页版 (免费)

本地部署: DeepSeek-V3-32B

API 开发: DeepSeek V3.2 API (最便宜)

Gemini 3 Pro：科研能力世界第一

Gemini 3 Pro 核心特点 (HLE 2025.12 数据)

- **HLE 科研能力**：37.5% (世界第一)
- **校准误差**：57 (置信度较准)
- **上下文**：2M tokens (业界最长)
- **多模态**：原生图文视频理解

为什么科研首选？

- HLE 测试中领先第二名 6 个百分点
- 可一次读入整本教科书或多篇论文
- 图表理解能力最强 (14% 多模态题)
- 与 Google Scholar/Workspace 深度集成

科研应用场景

- **文献综述**：2M 上下文读多篇论文
- **数据分析**：理解复杂图表
- **实验设计**：科学推理能力强
- **多模态研究**：图像/视频分析

访问方式

- Gemini App (免费版可用)
- Google AI Studio (开发者)
- Vertex AI (企业版)

科学研究模型选择指南

基于 Humanity's Last Exam (HLE) 2025.12 科研能力排名

研究任务	首选模型	备选方案
前沿科研问答	Gemini 3 Pro (37.5%)	GPT-5 Pro (31.6%)
博士级科学推理	Gemini 3 Pro	GPT-5.2 (27.8%)
代码开发	Claude Opus 4.5 (74.4%)	Gemini 3 Pro (74.2%)
本地部署科研	DeepSeek-V3-32B	Qwen 2.5-32B
预算敏感科研	DeepSeek R1 (8.5%)	Gemini Flash
多模态科学数据	Gemini 3 Pro	GPT-5.2
中文科学文献	DeepSeek V3.2	Qwen 3.0
长文档分析	Gemini 3 Pro (2M)	Claude Opus 4.5 (200K)

重要提醒

- 数据来源: scale.com/leaderboard/humanitys_last_exam (2025-12-17)

Gemini 3 Pro 在 HLE 科研测试中领先第二名近 6 个百分点

多模态能力：科研中的关键差异

什么是多模态能力？

- 处理**文本之外**的信息类型
- 图像、图表、公式、视频等
- 科研中经常需要分析复杂图表

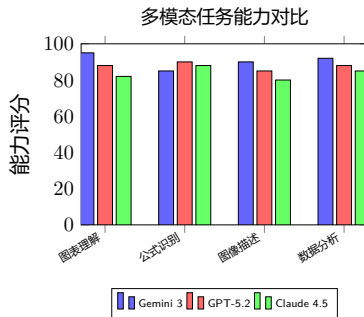
科研场景需求：

- **论文图表理解**：读懂实验结果图
- **公式识别**：从截图提取 LaTeX
- **数据可视化分析**：解读统计图表
- **实验数据**：显微镜/光谱图像

重要发现：

- HLE 测试中 **14%** 为多模态题目
- 多模态能力直接影响科研表现

主流模型多模态能力对比：



关键结论

Gemini 3 Pro：图表理解最强（原生多模态）

GPT-5.2：公式识别领先

Claude 4.5：多模态能力相对较弱

📄 场景 1：论文图表分析

Prompt 示例

请分析这张实验结果图：

[上传图片]

1. 描述主要趋势
2. 识别异常数据点
3. 总结统计意义

推荐：Gemini 3 Pro

- 理解复杂科学图表
- 准确提取数据趋势
- 支持图表对比分析

📄 场景 2：公式 OCR

应用

🔬 场景 3：科学图像识别

- 显微镜图像分析
- 光谱数据解读
- 晶体结构识别

推荐：专用模型 + Gemini 辅助

📊 模型选择速查表：

任务	推荐
图表/数据图	Gemini 3 Pro
公式识别	GPT-5.2
手绘草图	Gemini 3
PDF 截图	GPT-5.2
视频分析	Gemini 3 Pro

学术研究背景:

- Shah (2024): 《From Prompt Engineering to Prompt Science》
 - 提出将 Prompt 构建科学化
 - 强调可复现性和透明性
 - 人在环路 (Human-in-the-Loop) 验证
- Bsharat et al. (2024): 《26 Principled Instructions》
 - 提出 26 条 Prompt 设计原则
 - 在 GPT-3.5/4, LLaMA 上验证有效

核心观点

科研中使用 LLM 需要:

- 系统性的 Prompt 构建方法
- 可复现的实验流程
- 透明的决策过程
- 严格的效果验证

参考: arXiv:2401.04122, arXiv:2312.16171

经典 Prompt 技术：学术研究验证

1. Zero-shot Prompting

- 直接描述任务，无需示例
- 适用：简单明确的任务
- 优点：快速、通用

2. Few-shot Prompting

- 提供 2-5 个输入输出示例
- 适用：格式化输出、分类任务
- 研究表明：3-5 个示例最优

3. Chain-of-Thought (CoT)

- Wei et al. (2022) 提出
- 引导模型逐步推理
- 数学/逻辑任务提升显著
- 关键词： "Let's think step by step"

4. Self-Consistency

- 多次采样取多数结果
- 提升推理任务可靠性
- 适合关键决策场景

参考：Wei et al. "Chain-of-Thought Prompting" (NeurIPS 2022); Wang et al. "Self-Consistency" (ICLR 2023)

26 条 Prompt 设计原则精选

指令清晰度原则:

- ① 无需礼貌用语，直接表达
- ② 明确指定目标受众
" 为专业研究人员解释..."
- ③ 将复杂任务分解为子任务
- ④ 使用肯定指令，避免否定
" 做 X" 优于 " 不要做 Y"
- ⑤ 使用引导词: "Think step by step"

格式控制原则:

- ⑥ 使用分隔符区分内容
如 " ", <>, [TAG]
- ⑦ 指定输出格式和长度
- ⑧ 要求结构化输出 (JSON/表格)

角色与上下文原则:

- ⑨ 分配专家角色
" 你是 XX 领域资深专家..."
- ⑩ 提供背景和上下文信息
- ⑪ 使用 Few-shot 示例引导

质量提升原则:

- ⑫ 要求详细解释推理过程
- ⑬ 允许模型提问以澄清需求
- ⑭ 要求模型自我验证答案
- ⑮ 使用 " 我会给 \$X 小费 " 提升质量
(研究表明确有效果)

完整 26 条: [arXiv:2312.16171](https://arxiv.org/abs/2312.16171)

RAG (检索增强生成)

- Lewis et al. (2020) 提出
- 结合外部知识库检索
- 显著减少幻觉
- 科研应用：文献综述、数据查询

Tree of Thoughts (ToT)

- Yao et al. (2023) 提出
- 探索多条推理路径
- 适用：复杂问题求解
- 科研应用：实验设计、方案对比

参考: promptingguide.ai (DAIR.AI)

ReAct (推理 + 行动)

- Yao et al. (2023) 提出
- 交替推理与工具调用
- 适用：需要外部信息的任务
- 科研应用：数据分析、代码调试

Prompt Chaining

- 将复杂任务串联多个 Prompt
- 每步输出作为下一步输入
- 科研应用：论文写作流水线
 - 大纲 → 章节 → 润色 → 审校

科研 Prompt 最佳实践

医学/生物领域研究发现 (Zaghir et al. 2024)

- 系统性综述分析了医学领域 Prompt Engineering 应用
- 发现：结构化 Prompt 显著提升临床文本理解准确率
- 推荐：明确任务边界、提供领域术语定义

科学模拟与自动化 (Liu et al. 2024)

- 《Automated Simulation Research Workflow through LLM Prompt Engineering》
- 展示 LLM 驱动的科研自动化工作流
- 关键：迭代式 Prompt 优化 + 错误反馈循环

科研 Prompt 黄金法则

明确角色 → 提供上下文 → 结构化输出 → 要求解释 → 迭代优化

第三部分：AI 编程工具

什么是 AI Agent (智能体)?

传统大模型 vs Agent:

- **传统模型**: 单次问答, 被动响应
- **Agent**: 自主规划, 多步执行, 主动完成任务

Agent 的核心能力:

① 规划 (Planning)

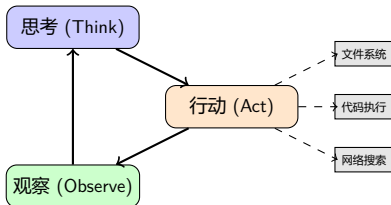
- 将复杂任务分解为子任务
- 制定执行计划和顺序

② 工具使用 (Tool Use)

- 调用外部 API 和工具
- 读写文件、执行代码

③ 记忆 (Memory)

- 保存执行历史和中间结果
- 从错误中学习并调整



ReAct 框架

Reasoning + Acting

Agent 在每一步都进行推理, 决定下一步行动, 并观察结果。

🧑 科研自动化场景:

① 文献综述自动化

- 搜索相关论文
- 提取关键信息
- 生成综述报告

② 数据分析流程

- 读取数据文件
- 执行统计分析代码
- 生成可视化图表

③ 代码调试与优化

- 分析错误信息
- 修改代码
- 重新运行测试

🔧 主流 Agent 工具:

• GitHub Copilot Agent

- 集成在 VS Code 中
- 自动修改多个文件
- 执行终端命令

• Claude Code (Anthropic)

- 专注于代码任务
- 支持完整项目开发

• OpenAI Codex/Jules

- 处理 GitHub Issue
- 自动化代码审查

MCP 协议

Model Context Protocol: 标准化 Agent 与工具的通信协议, 使不同模型可以调用相同的工具集。

Agent workflow示例：自动化论文分析

任务：分析一篇 PDF 论文并生成摘要

Agent 执行步骤：

① **Step 1: 规划**

- "我需要先读取 PDF，然后提取关键信息"

② **Step 2: 调用工具**

- 调用 PDF 解析工具读取文件

③ **Step 3: 分析**

- 识别研究问题、方法、结果

④ **Step 4: 生成输出**

- 调用写文件工具保存摘要

⑤ **Step 5: 验证**

- 检查输出是否符合要求

Agent 思考过程示例

[思考] 用户要分析论文，我需要：

1. 读取 PDF 内容
2. 提取核心信息
3. 生成结构化摘要

[行动] 调用 `read_pdf` 工具

参数: `{"path": "paper.pdf"}`

[观察] 获得论文文本...

[思考] 论文是关于深度学习的，
主要贡献是提出新架构...

[行动] 生成摘要并保存

与传统方式对比

传统：用户需要手动操作每一步

常用的编程智能体 (Programming Agents)

OpenAI Codex

- **简介:** GitHub Copilot 背后的核心动力, 专为代码生成优化的 GPT 模型。
- **特点:**
 - 经过数十亿行公开代码训练
 - 支持多种编程语言
 - 擅长将自然语言转化为代码
- **应用:** GitHub Copilot, OpenAI API

Google Jules

- **简介:** Google DeepMind 开发的编程智能体, 旨在处理复杂的编程任务。
- **特点:**
 - 具备多步推理能力
 - 能理解和修改大型代码库
 - 专注于解决 GitHub Issue 级别的任务
- **应用:** 集成于 Google 内部工具及 Gemini 生态

Agent vs. Assistant

- **Assistant (助手):** 补全代码, 回答问题 (如 Copilot 早期版本)。
- **Agent (智能体):** 自主规划, 执行多步操作, 调用工具, 解决复杂问题 (如 Jules, Copilot Workspace)。

为什么选择这个组合？

- 全球最流行的代码编辑器
- 微软官方 AI 助手
- 深度 IDE 集成
- 企业级安全保障

2026 年 1 月最新更新：

- 12/8: Coding Agent 支持模型选择
- 12/5: 代码生成指标仪表盘
- 12/4: GPT-5.1-Codex-Max 公测
- 12/3: Claude Opus 4.5 全平台支持
- 12/3: API 分配 Issue 给 Copilot
- 12/1: Copilot Spaces 公共空间

订阅价格 (2026 年 1 月)

Free	免费 (有限功能)
Pro	\$10/月
Pro+	\$39/月
Business	\$19/用户/月
Enterprise	\$39/用户/月

实用技巧

- 按 Tab 接受建议
- 按 Ctrl+Enter 查看多个建议
- 使用 @workspace 引用项目
- 使用 /fix 修复代码问题

1. 代码补全

```
# 只需输入注释, Copilot 自动生成
def calculate_fibonacci(n):
    # Copilot 会自动补全实现
    if n <= 1:
        return n
    return calculate_fibonacci(n-1) +
           calculate_fibonacci(n-2)
```

2. Copilot Chat

- @workspace - 项目上下文
- @vscode - 编辑器操作
- /explain - 解释代码
- /tests - 生成测试

3. Agent 模式 (最新)

- 自动执行多步骤任务
- 读取/修改多个文件
- 运行终端命令
- 创建 Pull Request

4. 模型选择 (12 月更新)

- GPT-5.1-Codex-Max
- Claude Opus 4.5
- Gemini 3 Pro
- 自动模型选择

Pro 技巧

在设置中启用 Agent 模式:

什么是 Cursor?

- 基于 VS Code 的 AI-native IDE
- 专为 AI 编程设计
- 100 万 + 活跃用户
- 被 Y Combinator、Stripe 等推荐

核心功能:

- **Tab**: 超智能代码补全
- **Multi-Agent**: 8 个 Agent 并行运行
- **Composer**: 自研 4x 快速模型
- **Browser**: 内置浏览器调试
- **Plan Mode**: 复杂任务规划

支持模型:

价格 (2026 年 1 月)

Hobby	免费
Pro	\$20/月
Pro+	\$60/月
Ultra	\$200/月
Teams	\$40/用户/月

最新更新 (v2.1 11/21)

- AI Code Review 编辑器内
- Instant Grep (即时搜索)
- Plan Mode 交互式问答
- Cloud Agent 99.9% 可靠性

什么是 Windsurf?

- 首个真正的“Agentic IDE”
- Codeium 公司出品
- 100 万 + 活跃用户
- 59% 财富 500 强企业使用

核心功能 - Cascade:

- 深度代码库理解
- 实时感知用户操作
- 智能上下文管理
- 自动修复 Linter 错误
- MCP 协议支持

亮点功能:

使用数据

- 每天 7000 万 + 行 AI 生成代码
- 94% 代码由 AI 编写
- JPMorgan Chase 创新奖

适用人群

- 前端开发者 (实时预览)
- 快速原型开发
- 企业级安全需求
- 需要本地部署

Claude Code - 终端编程神器

什么是 Claude Code?

- Anthropic 官方终端工具
- 直接在命令行中与 Claude 交互
- 已达到 **\$10 亿**收入里程碑

核心能力:

- 代码库快速上手 (几秒内理解整个项目)
- Issue 到 PR 一站式处理
- 多文件智能编辑
- 与 CLI 工具无缝集成
- 支持 GitHub/GitLab 集成

安装命令 (Windows):

```
irm https://claude.ai/install.ps1 | iex
```

包含在 Claude 订阅中

- Pro (\$20/月) - 短期任务
- Max 5x (\$100/月) - 日常使用
- Max 20x (\$200/月) - 重度用户

典型工作流

- ① 进入项目目录
- ② 运行 claude
- ③ 描述任务
- ④ Claude 自动分析、编辑、测试
- ⑤ 审核并提交

AI 编程工具对比

特性	Copilot	Cursor	Windsurf	Claude Code
类型	VS Code 插件	独立 IDE	独立 IDE	终端工具
基础价格	\$10/月	\$20/月	免费起	\$20/月起
Agent 能力	强	最强	强	强
模型选择	多模型	多模型	专有模型	Claude 系列
学习曲线	低	中	中	中
企业支持	最完善	有	强	有
离线使用	否	否	可选	否

选择建议

- **VS Code 忠实用户**: GitHub Copilot (无缝集成)
- **追求最佳 AI 体验**: Cursor (AI-native 设计)
- **前端开发/快速原型**: Windsurf (实时预览)
- **终端党/命令行爱好者**: Claude Code
- **预算有限**: Windsurf 免费版 + DeepSeek

GitHub Copilot 高级技巧 (一): 自定义指令

Custom Instructions 设置:

- 打开 VS Code 设置
- 搜索 copilot instructions
- 添加项目级或全局指令

指令文件

`.github/copilot-instructions.md`:

```
# Project Instructions for Copilot

## Code Style
- Use Python 3.11+ features
- Follow PEP8 strictly
- Always add type hints
- Prefer dataclasses over dicts

## Libraries
- Use PyTorch for deep learning
- Use pytest for testing
- Use black for formatting

## Conventions
- Function names: snake_case
```

使用场景:

团队规范统一

将 `.github/copilot-instructions.md` 提交到仓库, 团队成员 Copilot 自动遵循相同规范

高级指令示例:

```
## Scientific Computing
- Use numpy for array operations
- Always handle NaN values
- Include units in variable names
  (e.g., distance_meters)
- Add docstrings with parameter
  types and return values

## Paper-specific
- Reference equations using LaTeX
- Use consistent notation with paper
```

Pro 技巧

GitHub Copilot 高级技巧 (二): Agent 模式深度使用

启用 Agent 模式:

- 设置中启用
`github.copilot.chat.agent.enabled`
- 或使用 VS Code Insiders 版本

Agent 核心能力:

- **多文件编辑**: 一次修改多个相关文件
- **终端执行**: 自动运行命令验证
- **迭代修复**: 发现错误自动重试
- **上下文理解**: 理解整个项目结构

Agent Prompt 示例:

```
@workspace 请为这个项目添加完整的
pytest测试套件, 包括:
1. tests/目录结构
2. conftest.py配置
```

实用 Agent 命令:

代码重构

"@workspace 将所有 `print` 语句替换为 `logging`, 配置合适的 `log` 级别, 确保现有功能不变"

文档生成

"@workspace 为所有公共 API 生成 `docstring`, 使用 Google 风格, 包含参数类型和示例"

Bug 修复

"@workspace /fix 运行 `pytest` 发现 3 个失败用例, 请分析原因并修复"

Agent 模式 vs 普通 Chat

普通 Chat: 只能建议, 需手动执行

GitHub Copilot 高级技巧 (三): 上下文管理

上下文引用符号:

- `@workspace` - 整个项目
- `@file:path/to/file.py` - 特定文件
- `#selection` - 当前选中代码
- `#editor` - 当前打开文件
- `#terminalLastCommand` - 最近终端命令
- `#terminalSelection` - 终端选中内容

多文件上下文示例:

参考 `@file:models/base.py` 中的 `BaseModel` 类, 在 `@file:models/vit.py` 中实现一个 `Vision Transformer`, 保持相同的接口风格

Slash Commands 速查:

命令	功能
<code>/explain</code>	解释代码
<code>/fix</code>	修复问题
<code>/tests</code>	生成测试
<code>/doc</code>	生成文档
<code>/optimize</code>	优化性能
<code>/new</code>	创建新文件
<code>/newNotebook</code>	创建 Jupyter
<code>/clear</code>	清除对话

高效组合

`@workspace /tests` 为 `@file:src/utils.py` 中所有函数生成 `pytest` 测试, 覆盖边界情况

科研项目指令模板:

```
# .github/copilot-instructions.md

## Research Project: [Your Paper Title]

### Notation (from paper)
- X: input tensor, shape (B, C, H, W)
- y: labels, shape (B,)
- theta: model parameters
- L: loss function (cross-entropy)

### Code Standards
- NumPy docstring format
- Include paper equation references
- Use LaTeX in comments for math
- Type hints required

### Reproducibility
- Set random seeds explicitly
- Log all hyperparameters
- Save model checkpoints with config

### File Structure
- models/: neural network definitions
- data/: dataset loaders
```

实验代码生成 Prompt:

论文算法实现

" 根据论文公式 (3) 中的损失函数:
$$L = -\sum(y \cdot \log(p)) + \lambda ||W||_2$$

实现 PyTorch 版本, 包括:

1. 正则化项可配置
2. 数值稳定性处理
3. 支持类别权重"

与论文符号保持一致

在指令中定义论文符号后, Copilot 会自动使用相同的变量命名:

- 论文用 θ $\rightarrow$ 代码用 theta
- 论文用 $\mathcal{L}$ $\rightarrow$ 代码用 loss
- 论文用 $\hat{y}$ $\rightarrow$ 代码用 y_pred

第四部分：科研应用实践

AI 辅助论文写作

文献综述:

- 使用 Claude/GPT 总结多篇论文
- 生成文献对比表格
- 识别研究空白

论文撰写:

- 结构化大纲生成
- 段落润色和改写
- 学术语言优化
- LaTeX 公式辅助

推荐工具组合:

- ① Claude Max (长文本处理)
- ② ChatGPT + 联网搜索

实用 Prompt 示例

请帮我总结以下 5 篇论文的主要贡献，并以表格形式对比它们的方法、数据集和实验结果：

请将这段中文摘要改写成学术英语，保持专业性但提高可读性：

请帮我用 LaTeX 写出这个损失函数的数学公式，并解释每个符号的含义：

注意事项

- 始终核实 AI 生成的引用
- 保持学术诚信
- 作为辅助而非替代

AI 辅助数据分析与可视化

数据处理:

- 自动生成数据清洗代码
- 统计分析脚本
- 异常值检测

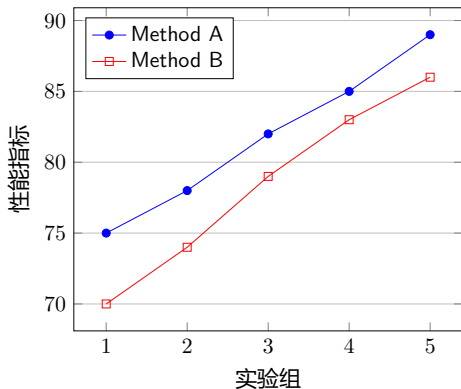
可视化:

- Matplotlib/Seaborn 代码生成
- 交互式图表 (Plotly)
- 论文级别图表美化

最佳实践:

- ① 用 Cursor/Copilot 生成基础代码
- ② 用 Claude 优化和解释
- ③ 用 ChatGPT Code Interpreter 验证

AI 生成的示例图表



Prompt 示例

用 Python matplotlib 生成一个对比我们方法和 baseline 的折线图, 要求论文发表级别质量

典型 workflow:

- ① **架构设计**: 用 Claude 讨论整体方案
- ② **代码生成**: 用 Cursor/Copilot 实现
- ③ **调试优化**: 用 Agent 模式自动修复
- ④ **文档生成**: 自动生成 docstring 和 README

深度学习场景:

- PyTorch/TensorFlow 模型代码
- 训练循环和验证逻辑
- 超参数搜索脚本
- 实验日志和可视化

高效技巧

- 提供清晰的输入输出规范
- 附上相关论文的伪代码
- 指定使用的库版本
- 要求生成单元测试

Agent 模式应用

@workspace 请帮我实现论文 X 中的算法 Y, 参考现有的 `model.py` 结构, 并添加相应的测试用例

CRISPE 框架:

- Capacity - 角色定义
- Request - 任务说明
- Input - 输入数据
- Steps - 步骤指引
- Personalization - 个性化
- Expectation - 期望输出

编程场景优化:

- 指定编程语言和版本
- 说明代码风格偏好
- 提供示例输入输出
- 要求添加注释

好的 Prompt 示例

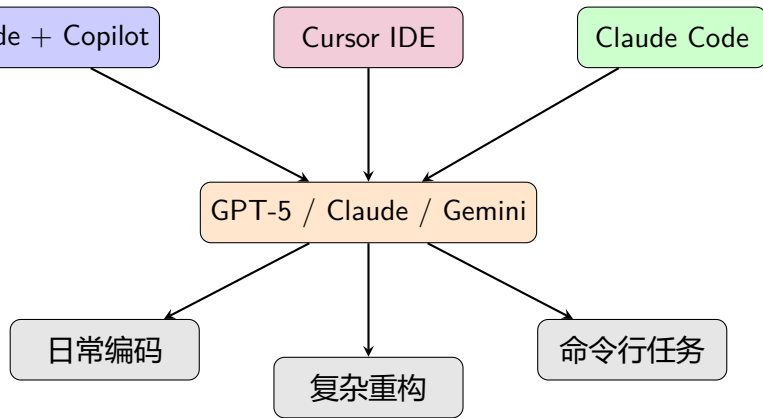
你是一位 Python 专家。请帮我实现一个高效的 LRU 缓存类，要求：

- 使用 Python 3.10+
- 时间复杂度 $O(1)$
- 包含类型注解
- 附带 pytest 测试
- 添加详细 docstring

避免的做法

- 模糊的描述
- 没有上下文

工具链整合建议



推荐组合

- **入门用户:** VS Code + GitHub Copilot Free

常见问题与解决方案

Q1: 生成的代码有 bug 怎么办?

- 使用 Agent 模式自动修复
- 提供错误信息让 AI 诊断
- 要求生成测试用例验证

Q2: 如何处理大型代码库?

- 使用 @workspace 引用上下文
- 分模块逐步处理
- 使用 Cursor 的索引功能

Q3: 如何保护代码隐私?

- 选择企业版 (数据不训练)
- 考虑本地部署选项
- 使用 Windsurf 企业版

Q4: 模型选择困难?

- 启用自动模型选择
- 简单任务用轻量模型
- 复杂任务用旗舰模型

Q5: 如何控制成本?

- 使用缓存功能
- 合理选择模型级别
- 批量处理降低调用次数
- 利用免费额度

Q6: 中文支持不好?

- 优先考虑 DeepSeek
- Claude 中文表现也不错

第五部分：高级技巧

MCP 协议：让 AI 连接真实世界

什么是 MCP (Model Context Protocol)?

- Anthropic 提出的开放协议
- 让 AI 直接操作本地文件、数据库、API

支持的工具：

- Claude Desktop / Claude Code
- Cursor IDE (原生支持)
- Windsurf
- VS Code Copilot (部分支持)

科研应用场景：

- 直接读取/分析本地数据文件
- 自动执行实验脚本
- 连接文献数据库 (Zotero, Mendeley)

常用 MCP 服务器

- filesystem - 文件操作
- github - 代码仓库
- postgres/sqlite - 数据库
- brave-search - 网络搜索
- arxiv - 论文检索

配置示例

在 `claude_desktop_config.json` 中添加 MCP 服务器配置，即可让 Claude 直接访问本地资源。

多模型协作工作流 (Multi-Agent Workflow)



各模型分工建议

任务阶段	推荐模型	理由
文献阅读/长文本	Gemini 3 Pro	2M 超长上下文，多模态理解
中文任务/数学推理	DeepSeek V3.2	中文最强，数学能力顶尖
代码实现	Claude Opus 4.5	代码质量最高，Agent 能力强
论文润色/写作	GPT-5.2 / Claude	语言流畅，学术风格好
图表理解/修改	Gemini 3 Pro	多模态能力最强

AI 辅助论文自查清单 (Pre-submission Checklist)

让 AI 扮演审稿人:

Prompt (English)

"Act as a critical reviewer for [Conference/Journal]. Review my paper and identify: 1. Weak claims without sufficient evidence; 2. Missing related work; 3. Unclear methodology; 4. Overclaiming in the conclusion; 5. Grammar and style issues. Be harsh but constructive."

检查清单:

- ☐ 摘要是否准确概括贡献?
- ☐ Related Work 是否全面?
- ☐ 方法描述是否可复现?

分段检查 Prompt:

Abstract 检查

"Does this abstract clearly state: 1. The problem; 2. Our approach; 3. Key results; 4. Main contribution? Suggest improvements."

实验检查

"Review my experimental section. Check: 1. Are baselines fair and up-to-date? 2. Are datasets appropriate? 3. Is statistical significance reported? 4. Are ablation studies sufficient?"

重要提醒

常见“坑”与避坑指南

1. AI 幻觉 (Hallucination)

- **问题**: AI 编造不存在的论文、数据
- **解决**: 要求提供来源, 交叉验证

2. 引用编造

- **问题**: 生成看起来真实但不存在的引用
- **解决**: 永远在 Google Scholar 验证

3. 代码看似正确实则有 bug

- **问题**: 逻辑错误、边界条件遗漏
- **解决**: 要求生成单元测试

4. 过度自信的错误答案

- **问题**: AI 用自信的语气说错话
- **解决**: 追问“Are you sure?”

5. 上下文遗忘

- **问题**: 长对话后忘记早期内容
- **解决**: 定期总结, 使用长上下文模型

6. 隐私泄露

- **问题**: 敏感数据被发送到云端
- **解决**: 使用企业版或本地模型

本地部署开源模型指南

为什么需要本地部署？

- 处理敏感/保密数据
- 无网络环境
- 降低长期成本
- 完全的数据控制

推荐开源模型 (2026.01):

- Llama 3.2 (Meta) - 通用能力强
- Qwen 2.5/3.0 (阿里) - 中文最佳开源
- DeepSeek-Coder - 代码专精
- Mistral Large - 欧洲之光

部署工具:

- ollama - 最简单，一键部署

硬件要求参考

模型大小	显存需求
7B 参数	8GB VRAM
13B 参数	16GB VRAM
70B 参数	48GB+ VRAM

Ollama 快速开始

```
# 安装 Ollama
winget install Ollama.Ollama

# 运行 Qwen 中文模型
ollama run qwen2.5:7b
```


下一步行动

立即可以做的事情

- ① **今天**: 安装 VS Code + GitHub Copilot (免费版即可开始)
- ② **本周**: 尝试用 AI 辅助完成一个小任务
- ③ **本月**: 比较 Cursor 和 Copilot, 选择适合自己的
- ④ **持续**: 关注模型更新, 保持学习

推荐资源:

- GitHub Copilot 官方文档
- Cursor 官方教程
- Claude 提示词工程指南
- DeepSeek 开发者文档

学习社区:

- GitHub Discussions
- Cursor Discord
- Reddit r/ChatGPT
- 知乎 AI 编程话题

使用技巧：如何获得更详细的回答

为什么 AI 回答有时很简短？

- 大模型通过预测“下一个词”生成回答
- 训练数据决定了**平均输出长度**
- 遇到“结束标记 (EOF)”就停止输出
- 模型倾向于给出“够用”的回答

获取详细回答的技巧：

- “**详细一点**” - 最简单有效
- “**请展开说明**” - 要求更多细节
- “**请给出完整示例**” - 要求完整代码/步骤
- “**请分步骤解释**” - 结构化输出
- “**至少写 1000 字**” - 明确长度要求

大模型输出原理

大模型是**逐词预测**的：

- 根据上文预测下一个词的概率
- 通过采样选择输出哪个词
- 重复直到生成结束标记

因此，**提示词会影响输出长度！**

示例对比

简单问法：“解释什么是机器学习”
→ 可能只得到 2-3 句话

详细问法：“详细解释什么是机器学习，包括定义、主要类型、应用场景和学习建议”
→ 会得到完整详细的回答

ChatGPT 订阅 vs API 计费——两种完全不同的模式

👤 普通用户: ChatGPT 订阅制

计费方式: 按月订阅 (一口价)

- **Free**: 0 元 (基础功能)
- **Plus**: \$20/月
- **Pro**: \$200/月 (极高频用户)

限制方式: 按次数 (Messages)

- 限制你“点击发送”的次数
- 如: Plus 用户每 3 小时约 80 次
- 无论回答长短, 都算 1 次
- 不用担心生成字数多会多扣钱

判断方法

每月固定扣费, 用不用都扣这个钱

</> 开发者/第三方应用: API 计费

计费方式: 按 Token 精确计费

- **输入 Token**: 发送的内容 (含历史)
- **输出 Token**: AI 回答的内容
- 输出通常比输入贵 (如 4 倍)

限制方式: 用多少付多少

- 需要填入 API Key (sk-xxxx)
- 适用于: 沉浸式翻译、写作软件等
- 生成字数越多, 费用越高

判断方法

需要充值, 看着余额减少

避坑指南：警惕“套壳”应用

⚠️ 常见套路

- 很多第三方 AI 网站/APP
- 声称“按次收费”或“按月不限次”
- 实际是调用官方 API 后**加价倒卖**

套壳应用的计费真相：

- 他们按 Token 从官方购买服务
- 然后按次/按月卖给你
- 通常存在**隐藏限制**
- 可能**加价 50-200%**

识别方法

- 不是官方域名（如非 chatgpt.com）

✅ 推荐做法

日常使用

- 直接用官方 ChatGPT/Claude
- 订阅制，次数限制清晰
- 数据安全有保障

开发/集成

- 直接申请官方 API Key
- 或使用可信中转（如云雾 AI）
- 按 Token 付费，透明计价

官方渠道：

- OpenAI: chatgpt.com

API 省钱技巧 (一): 免费额度汇总

完全免费的服务:

- ★ DeepSeek - 网页端无限免费
- ★ Kimi - 长文本处理免费
- ★ 通义千问 - 基础功能免费
- ★ Google AI Studio - Gemini 免费额度
- ★ GitHub Copilot Free - 有限代码补全

API 免费额度:

- OpenAI: 新用户 \$5 额度 (3 个月有效)
- Anthropic: \$5 免费额度
- Google: Gemini API 免费层
- DeepSeek: 100 万 Token/月免费
- 阿里云 Qwen: 免费调用额度

学生/教育优惠:

- GitHub Education Pack**
 - 免费 Copilot Pro (价值 \$10/月)
 - 需 .edu 邮箱认证
- Azure for Students**
 - \$100 免费 Azure 额度
 - 可用于 Azure OpenAI
- Google Cloud**
 - 学生 \$300 免费额度

省钱核心原则

日常用免费服务, 关键任务用付费 API, 学会组合使用!

API 省钱技巧 (二): Token 计费原理

什么是 Token?

- 模型处理文本的基本单位
- **英文**: 约 4 字符 = 1 Token
- **中文**: 约 1-2 字 = 1 Token
- 1000 Token $\approx$ 750 英文单词
- 1000 Token $\approx$ 500-700 中文字

计费方式:

- **输入 Token**: 你发送的内容
- **输出 Token**: AI 返回的内容
- 通常输出比输入贵 2-4 倍
- 按百万 Token (M Token) 计价

主流模型价格对比 (输入/输出):

模型	输入	输出
GPT-5.2	\$5/M	\$15/M
GPT-5.2-mini	\$0.3/M	\$1.2/M
Claude Opus 4.5	\$20/M	\$80/M
Claude Sonnet 4	\$4/M	\$16/M
Gemini 3 Pro	\$2/M	\$8/M
DeepSeek V3.2	\$0.4/M	\$1.6/M

省钱计算

处理 10 篇论文 (约 50 万 Token):

GPT-5.2: $\approx$ \$10

DeepSeek V3.2: $\approx$ \$1.0 (**省 90%!**)

API 省钱技巧 (三): 优化策略

1. 缓存策略

- 相同问题不重复调用
- 本地存储常用回答
- 使用 Redis/SQLite 缓存
- Anthropic 原生支持 Prompt 缓存

2. Prompt 优化

- 精简 System Prompt
- 避免重复上下文
- 使用 few-shot 而非长说明
- 及时清理对话历史

3. 模型选择策略

- 简单任务用小模型 (mini/baiku)

许达 (未来院三室)

4. 批量处理 vs 单次调用

单次调用

每次请求都带完整 System Prompt
10 次调用 = 10 次 System Prompt 费用

批量处理

一次请求处理多个任务
1 次调用 = 1 次 System Prompt 费用
节省 70-90% 成本!

5. 使用中转服务

- 云雾 AI 等中转价格更优
- 支持多模型切换
- 适合国内开发者

绝对不要发送的数据：

- × 未发表的核心研究成果
- × 专利申请中的技术细节
- × 公司机密/商业秘密
- × 个人隐私信息（身份证、银行卡）
- × 患者医疗数据
- × 学生成绩等敏感教育数据
- × 未脱敏的用户数据

数据脱敏技巧：

- 用占位符替换敏感信息
- 只发送代码结构，不发数据
- 使用假数据测试

企业版 vs 个人版数据政策：

特性	个人版	企业版
数据用于训练	可能	否
数据保留	30 天 +	可配置
SOC2 认证	否	是
数据加密	传输中	全程
审计日志	无	有

本地部署 vs 云端服务：

本地部署优势

数据完全本地、无网络依赖、长期成本低

云端服务优势

模型更强、无需硬件、即开即用

第六部分：风险与规范

AI 生成内容检测工具概览

主流 AI 检测工具:

- **Turnitin AI Detection**
 - 学术界最权威
 - 已集成至多数高校系统
 - 检测准确率约 98%
- **GPTZero**
 - 专为教育场景设计
 - 提供逐句分析
 - 免费版可用
- **Originality.AI**
 - 商业级检测
 - 支持批量检测

检测原理:

统计特征分析

- **困惑度 (Perplexity)**: AI 文本通常更“可预测”
- **突发性 (Burstiness)**: 人类写作句长变化大
- **词汇多样性**: AI 倾向使用常见词

重要提醒

- 检测工具存在**假阳性**风险
- 非母语者文本更易被误判
- 技术文档/代码注释误判率高

明确允许 (需声明):

- ✓ **Nature 系列**
 - 必须在 Methods 或致谢中声明
 - AI 不能作为作者
- ✓ **Science 系列**
 - 允许用于润色和编辑
 - 需明确说明使用方式
- ✓ **ACL/EMNLP/NeurIPS**
 - 允许辅助写作
 - 禁止 AI 直接生成核心内容

标准声明模板:

推荐写法

"During the preparation of this work, the authors used [Tool Name] for [specific purpose, e.g., language editing, code debugging]. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication."

禁止的行为:

- × AI 作为论文作者
- × 未声明的大规模 AI 生成
- × AI 生成数据/实验结果
- × AI 生成文献引用

如何合规使用 AI 避免学术不端

✓ 合规使用场景:

- 语法和拼写检查
- 语言润色（非母语者）
- 代码调试和优化
- 文献检索辅助
- 格式转换（如 Markdown→LaTeX）
- 数据可视化代码生成
- 解释复杂概念（学习用途）

× 禁止使用场景:

- 生成核心创新内容
- 编造实验数据
- 生成虚假引用
- 代写整篇论文

降低 AI 检测风险的技巧:

合法优化方法

- 1 **分段处理**：不要一次性生成成长文本
- 2 **人工改写**：AI 输出后大幅修改
- 3 **混合写作**：AI 润色人写的草稿
- 4 **增加个性**：加入个人观点和例子
- 5 **使用引用**：增加具体文献支撑

黄金法则

AI 是助手，不是作者

你必须能解释和捍卫论文中的每一句话

学术诚信自查清单

提交前必查项目：

- ☐ 所有引用已在数据库中验证存在
- ☐ 实验数据来自真实实验
- ☐ 核心创新点是原创思考
- ☐ AI 使用已按期刊要求声明
- ☐ 能够解释论文任何部分
- ☐ 合作者都同意最终版本
- ☐ 没有一字不改地使用 AI 输出

记录保留建议：

- 保存所有 AI 对话记录
- 保留论文各版本草稿
- 记录 AI 使用的具体环节

被质疑时的应对：

如果收到 AI 检测警告

- ① **保持冷静：**检测工具有误判
- ② **提供证据：**展示写作过程记录
- ③ **解释内容：**证明你理解所写内容
- ④ **展示草稿：**提供修改历史

预防措施

- 使用 Git 管理论文版本
- 用 Overleaf 等协作平台（有历史记录）
- 定期保存写作进度截图

科研绘图免费工具大全（一）：流程图与架构图

1. Mermaid + AI

- **完全免费开源**
- 用文本描述生成图表
- 支持流程图、时序图、类图等
- **集成**：VS Code、GitHub、Notion

让 AI 生成 Mermaid 代码

" 请用 Mermaid 语法画一个神经网络训练流程图，包括数据加载、前向传播、损失计算、反向传播、参数更新"

2. Draw.io (diagrams.net)

- **完全免费**，无需注册
- 丰富的科学图形库
- 支持导出 PDF/SVG/PNG
- 网页版：draw.io

3. Excalidraw

- **开源免费**，手绘风格
- 适合 PPT 和博客配图
- 有 AI 辅助插件
- excalidraw.com

4. PlantUML

- **开源**，文本转 UML 图
- 支持类图、活动图、状态图
- 可与 LaTeX 集成

AI + 绘图 workflow

让 ChatGPT/Claude 生成 Mermaid 或 PlantUML 代码 → 粘贴到在线编辑器 → 导出高清图片

科研绘图免费工具大全（二）：数据可视化

1. Python + Matplotlib/Seaborn

- 完全免费开源
- 科研论文标准工具
- AI 可直接生成代码

Prompt 示例

" 用 matplotlib 画一个双 Y 轴图，左轴是 accuracy，右轴是 loss，X 轴是 epoch，使用 Science 期刊配色 "

2. Plotly (开源版)

- 交互式图表
- 支持 3D 可视化
- 可导出静态图片

3. Altair

4. Python 绘图配色推荐：

科研级配色方案

- scienceplots - Nature/Science 风格
- seaborn - 统计图表配色
- cmocool - 海洋学配色
- palettable - 多种学术配色

5. 在线工具

- RAWGraphs - 开源数据可视化
- Flourish - 免费版可用
- Datawrapper - 新闻级图表

安装科研绘图包

`pip install scienceplots`

科研绘图免费工具大全 (三): 生物医学与专业图

1. BioRender 替代方案 (免费)

- **Bioicons** (bioicons.com)
 - 免费生物图标库
 - SVG 格式可编辑
- **SciDraw** (scidraw.io)
 - 开源科学插图
 - 可商用
- **Servier Medical Art**
 - 免费医学插图库
 - CC BY 3.0 许可

2. 化学结构图

- **ChemDraw Free** (在线版)
- **MolView** - 开源分子可视化
- **RDKit** - Python 化学库

3. 神经网络架构图

- **NN-SVG** (alexlenail.me/NN-SVG)
 - 免费在线工具
 - 生成精美神经网络图
- **Netron** - 模型可视化
- **TensorBoard** - 训练可视化

4. LaTeX 绘图

- **TikZ/PGFPlots**
 - 论文内嵌图表最佳
 - AI 可生成 TikZ 代码
- **pgfgantt** - 甘特图
- **circuitikz** - 电路图

AI 辅助绘图 Prompt 模板

生成 Matplotlib 代码:

论文级图表

"Generate Python code for a publication-quality figure:

- Data: [your data]
- Type: line plot with error bars
- Style: Nature journal (use scienceplots)
- Font: Times New Roman, 12pt
- Size: 3.5 inch width (single column)
- DPI: 300
- Save as PDF and PNG"

生成 TikZ 代码:

流程图

"用 TikZ 画一个机器学习 pipeline 流程图, 包括: 数据预处理 → 特征提取 → 模型训练 → 评估, 使用蓝色系配色"

生成 Mermaid 代码:

系统架构图

"用 Mermaid 画一个微服务架构图, 包括: API Gateway, User Service, Order Service, Database, 用 flowchart TB 格式"

免费工具速查表:

用途	推荐工具
流程图	Draw.io, Mermaid
数据图表	Matplotlib, Plotly
架构图	Excalidraw, PlantUML
生物医学	Bioicons, SciDraw
神经网络	NN-SVG, TikZ
化学结构	MolView, RDKit

科研 LLM 应用的关键风险

1. 幻觉与事实错误

- 编造不存在的引用
- 虚构实验数据
- 错误的数学推导

2. 可复现性问题

- 模型输出随机性
- API 版本变化
- Prompt 敏感性

3. 学术诚信风险

- 过度依赖 AI 写作
- 未声明 AI 使用
- 抄袭检测新挑战

4. 偏见传播

- 训练数据偏见
- 领域知识不均衡
- 语言/文化偏见

FLAWS 基准发现 (Xi et al. 2025)

LLM 在科学论文错误检测中:

- 能检测明显错误
- 但细微逻辑错误难以发现
- 过度自信问题普遍

arXiv:2511.21843

写作阶段:

- ☐ 所有引用人工验证
- ☐ 数据和结果人工核实
- ☐ 声明 AI 辅助使用
- ☐ 保持原创核心贡献

代码生成:

- ☐ 运行并验证所有代码
- ☐ 添加单元测试
- ☐ 检查边界情况
- ☐ 代码审查

数据分析:

- ☐ 验证统计方法正确性
- ☐ 检查数据预处理逻辑
- ☐ 人工复核关键结果
- ☐ 保存完整分析日志

核心原则

AI 辅助，人类负责

所有发表内容由作者承担责任

记录要求:

① 模型信息

- 模型名称和版本
- API 调用日期
- 温度等参数设置

② Prompt 记录

- 完整 Prompt 文本
- System prompt
- 上下文信息

③ 输出记录

- 原始输出保存
- 人工修改追踪
- 版本控制

论文方法部分示例

"We used GPT-5.2 (version 2025-11) with temperature=0.3 for code generation. The prompts are provided in Appendix A. All generated code was manually reviewed and tested before inclusion."

开放科学建议

- 发布 Prompt 模板
- 共享验证脚本
- 记录失败案例
- 讨论局限性

科研 Prompt 模板 (一): LaTeX 公式生成

基础公式生成:

Prompt 模板

请将以下数学表达式转换为 LaTeX 格式:

[你的公式描述]

要求:

1. 使用 `equation` 环境
2. 添加公式编号标签
3. 符号要有注释说明

示例输出:

```
\begin{equation}
\label{eq:loss}
\mathcal{L} = -\sum_{i=1}^N
y_i \log(\hat{y}_i)
\end{equation}
% where:
% $\mathcal{L}$: cross-entropy loss
% $N$: number of samples
% $y_i$: ground truth label
% $\hat{y}_i$: predicted probability
```

复杂公式推导:

Prompt 模板

请用 LaTeX 写出从公式 A 推导到公式 B 的完整过程:

起始: [公式 A]

目标: [公式 B]

使用 `align` 环境, 每步加注释

技巧

- 指定使用的宏包 (amsmath 等)
- 要求符号与论文其他部分一致
- 让 AI 解释每个符号的含义

科研 Prompt 模板 (二): Related Work 撰写

文献总结 Prompt:

模板

请阅读以下论文摘要, 为我的 Related Work 部分撰写一段总结:

[粘贴 3-5 篇论文摘要]

要求:

1. 按时间/方法类型分组
2. 指出各方法的优缺点
3. 引用用 [Author, Year] 格式
4. 最后指出研究空白
5. 学术英语, 客观语气

对比表格生成:

模板

根据以下论文, 生成一个方法对比表格 (LaTeX tabular 格式):

列: 方法名、数据集、指标、优点、缺点

注意事项:

重要!

- 永远不要让 AI 编造引用
- 只让 AI 总结你提供的文献
- 所有引用必须人工核实
- 在 Google Scholar 验证每篇论文

推荐工作流:

- ① 在 Semantic Scholar/Google Scholar 找论文
- ② 复制摘要给 AI 总结
- ③ AI 生成初稿
- ④ 人工核实和润色

科研 Prompt 模板 (三): 实验结果分析

数据分析 Prompt:

模板

请分析以下实验结果:

[粘贴数据表格或 CSV]

要求:

1. 指出主要发现
2. 比较不同方法的性能差异
3. 分析可能的原因
4. 指出异常值或有趣的模式
5. 建议后续实验方向

图表代码生成:

模板

用 Python matplotlib 生成论文级别图表:

数据: [你的数据]

要求: Nature/Science 风格, 300dpi,

Times New Roman 字体, 合适的图例位置

统计检验 Prompt:

模板

对以下两组数据进行统计显著性检验:

组 A: [数据]

组 B: [数据]

请: 1. 选择合适的检验方法

2. 报告 p 值和效应量

3. 给出学术写作中的标准表述

Ablation Study 分析:

模板

分析以下消融实验结果:

[消融实验表格]

写一段 300 字的分析, 说明

每个组件对最终性能的贡献

科研 Prompt 模板 (四): Rebuttal 撰写

回复审稿意见 Prompt:

模板

审稿人提出以下问题:

"[审稿意见原文]"

我们的回应要点:

[你的回应要点]

请帮我撰写正式的 Rebuttal 回复:

1. 首先感谢审稿人
2. 逐点回应, 礼貌但有力
3. 引用论文具体章节/图表
4. 说明我们做了哪些修改
5. 专业学术语气

常用回复句式:

感谢 + 回应模板

"We thank the reviewer for this insightful comment. We have addressed this concern by..."

补充实验模板

"Following the reviewer's suggestion, we conducted additional experiments..."
"The new results (Table X) demonstrate that..."

Rebuttal 黄金法则

- 永远保持礼貌和专业
- 直接回应, 不要绕弯子

研究验证的 Prompt 模式 (一): 结构化科研任务

假设生成 Prompt (基于 LLM4SR)

模板

【背景】

研究领域: [你的领域]
已知发现: [列出 3-5 个关键发现]
研究空白: [你识别的空白]

【任务】

生成 3 个可检验的研究假设:

1. 基于已有发现的延伸
2. 跨领域创新结合
3. 反直觉但有理论支撑

【要求】

- 每个假设明确可证伪
- 说明验证方法
- 预测可能结果

文献综述规划 Prompt

模板 (LitLLM 风格)

【输入】

主题: [你的研究主题]
关键词: [5-8 个关键词]

【任务】

Step 1: 生成文献检索策略
Step 2: 设计综述大纲 (5-7 节)
Step 3: 为每节列出应覆盖的问题

【输出格式】

JSON 结构, 包含:

- search_queries: 检索词组合
- outline: 大纲结构
- questions: 各节核心问题

研究验证的 Prompt 模式 (二): 代码生成最佳实践

科学计算代码 Prompt (基于实证研究)

Data-Aware 模板

【数据规格】

文件: `experiment_results.csv`

行数: 1000 条记录

列定义:

- `method`: `str` (A/B/C)
- `accuracy`: `float` [0,1]
- `runtime`: `float` (秒)
- `dataset`: `str` (5 个数据集名)

【任务】

生成统计分析代码:

1. 按 `method` 分组计算均值 $\pm$ std
2. 配对 `t` 检验 (A vs B, A vs C)
3. 生成论文 Table 1 格式输出

【约束】

- 使用 `pandas` + `scipy`

迭代修复模式

错误反馈 Prompt

代码执行出现以下错误:

```

[粘贴完整错误信息]

```

原始代码:

```python

[粘贴相关代码段]

```

请:

1. 诊断错误原因
2. 提供修复后的完整代码
3. 解释修改点

关键发现

研究验证的 Prompt 模式 (三): Chain-of-Thought 科研版

方法论推导 CoT

模板

【问题】

设计一个 [任务类型] 的方法

目标: [具体目标]

约束: [限制条件]

【推理步骤】

Step 1: 分析任务本质

- 输入是什么? 输出是什么?
- 核心挑战是什么?

Step 2: 回顾相关方法

- 现有方法如何解决?
- 各自的优缺点?

Step 3: 设计新方法

- 如何结合优点?
- 如何解决现有缺点?

Step 4: 验证可行性

许达 (未来院三室)

实验设计 CoT

模板

【研究问题】

验证假设 H: [你的假设]

【逐步设计】

Think step by step:

1. 需要什么数据?
 - 数据集选择理由
 - 样本量估算
2. 比较什么基线?
 - SOTA 方法列表
 - 公平性考虑
3. 用什么指标?
 - 主要指标 + 辅助指标
 - 统计显著性要求

研究验证的 Prompt 模式 (四): 多 Agent 协作科研

模拟同行评审

Reviewer Agent Prompt

你是 [顶会名称] 的资深审稿人。

【评审标准】

- Novelty (1-5): 创新性
- Soundness (1-5): 方法严谨性
- Clarity (1-5): 表达清晰度
- Significance (1-5): 重要性

【任务】

1. 给出各项评分和理由
2. 列出主要优点 (3 条)
3. 列出主要缺点 (3 条)
4. 提出具体改进问题 (5 个)
5. 给出最终建议:

Accept/Weak Accept/

Weak Reject/Reject

许达 (未来院三室)

Devil's Advocate 模式

反驳者 Prompt

扮演一个严格的批评者:

【论文声明】

[你的主要贡献声明]

【任务】

试图反驳这个声明:

1. 找出逻辑漏洞
2. 质疑实验设计
3. 提出反例场景
4. 指出过度声明

对每个批评点,
建议如何加强论述

研究支持

科研 LLM 工具生态系统：2026 年全景图

文献管理与检索

- Semantic Scholar - 语义论文搜索
- Elicit - AI 研究助手
- Consensus - 科学共识查询
- Connected Papers - 论文关系图
- ResearchRabbit - 文献推荐

写作与编辑

- Writefull - 学术语言优化
- Paperpal - 论文润色
- Grammarly - 语法检查
- QuillBot - 改写工具

代码与分析

- GitHub Copilot - 代码补全
- Cursor - AI-first IDE
- Code Interpreter - 数据分析
- Julius.ai - 可视化生成

专业领域工具

- BioGPT - 生物医学
- ChemCrow - 化学研究
- Galactica - 科学知识
- AlphaFold - 蛋白质结构

选择原则

任务专用工具 > 通用 LLM

科研 LLM 应用路线图：从入门到精通

Level 1: 基础应用

文献总结 | 语言润色 | 简单代码生成 | LaTeX 公式

Level 2: 系统应用

Related Work 撰写 | 数据分析流程 | 实验代码开发 | 审稿意见回复

Level 3: 高级应用

研究想法生成 | 多 Agent 协作 | 自动化工作流 | 跨领域探索

Level 4: 前沿应用

AI Co-scientist | 全流程自动化 | 科研 Agent 开发 | 领域定制模型

2026 年建议

大多数科研人员应达到 **Level 2-3**，重点是
高效使用现有工具而非追求最新技术

附录

主流开源模型

- **LLaMA 3.1** (Meta)

- 8B/70B/405B 参数
- 商用友好许可
- 多语言支持

- **Qwen2.5** (阿里巴巴)

- 0.5B-72B 多规格
- 中文能力出色
- Apache 2.0 许可

- **DeepSeek V3**

- MoE 架构, 高效推理
- 开源权重, MIT 许可
- 性能接近 GPT-4

特化模型

- **代码生成**: CodeLlama, StarCoder2
- **数学推理**: DeepSeek-Math
- **多模态**: LLaVA, Qwen-VL
- **医学**: BioMistral, Med-PaLM 开源版

选择建议

- 资源有限 → Qwen2.5-7B
- 中文任务 → Qwen2.5/DeepSeek
- 代码任务 → CodeLlama/StarCoder2
- 最强性能 → LLaMA 3.1-405B

附录 A：本地部署方案

推理框架

Ollama 最简单的本地部署

- 一键安装运行
- 支持主流开源模型
- Mac/Linux/Windows

vLLM 高性能推理

- PagedAttention 优化
- 高吞吐量服务
- 适合生产环境

llama.cpp CPU 推理

- 纯 C++ 实现
- 无需 GPU
- 量化支持

硬件需求估算

模型规模	显存 (FP16)	显存 (4bit)
7B	14GB	4GB
13B	26GB	8GB
70B	140GB	35GB

Ollama 快速开始

```
# 安装后直接运行
ollama run llama3.1

# 或运行其他模型
ollama run qwen2.5:7b
ollama run deepseek-v3
```

附录 A: vLLM 部署示例

安装与启动

安装vLLM

```
pip install vllm
```

启动API服务

```
python -m vllm.entrypoints.openai.api_server  
    --model Qwen/Qwen2.5-7B-Instruct \  
    --host 0.0.0.0 \  
    --port 8000
```

Python 调用

```
from openai import OpenAI
```

使用OpenAI兼容API

```
client = OpenAI(  
    base_url="http://localhost:8000/v1",  
    api_key="not-needed"  
)  
  
response = client.chat.completions.create(  
    model="Qwen/Qwen2.5-7B-Instruct",  
    messages=[{"role": "user",  
                "content": "你好"}]  
)
```

附录 A: WebUI 工具

Open WebUI

- 类 ChatGPT 界面
- 支持 Ollama 后端
- Docker 一键部署
- 多用户管理
- RAG 文档上传

```
docker run -d -p 3000:8080 \
-v open-webui:/app/backend/data \
```

```
ghcr.io/open-webui/open-webui:main
```

Text Generation WebUI

- Gradio 界面
- 多种加载器支持
- 参数调节丰富
- 扩展系统
- 适合实验调试

科研场景推荐

- 日常使用 → Open WebUI
- 模型调试 → Text Gen WebUI
- 批量处理 → vLLM API

附录 A：云端部署选项

主流云平台

平台	特点	适用场景	价格参考
Hugging Face	模型托管/Spaces	演示/小规模	免费-\$9/月
阿里云 PAI	国内主流	企业生产	按需计费
AutoDL	性价比高	学术研究	¥ 1-3/小时
Vast.ai	便宜 GPU	实验测试	\$0.2-1/小时

学术用户推荐

- ① AutoDL 租用 GPU 实例
- ② 配置好环境后保存镜像
- ③ 按需开机，用完关闭
- ④ 成本可控，灵活使用

免费资源

- Google Colab (有限 GPU)
- Kaggle Notebooks
- HF Spaces 免费层
- 各厂商新用户优惠

附录 A：数据隐私与合规

本地部署优势

- ✓ 数据完全本地化
- ✓ 无 API 调用记录
- ✓ 符合单位保密要求
- ✓ 可离线使用
- ✓ 无使用成本

适用场景

- 涉密/敏感数据处理
- 内部文档分析
- 无法联网的环境
- 高频调用需求

部署策略建议

分级使用策略：

① 公开数据：云端 API

- 使用最强模型
- 按需付费

② 内部数据：本地 7B 模型

- Ollama + Open WebUI
- 单卡即可运行

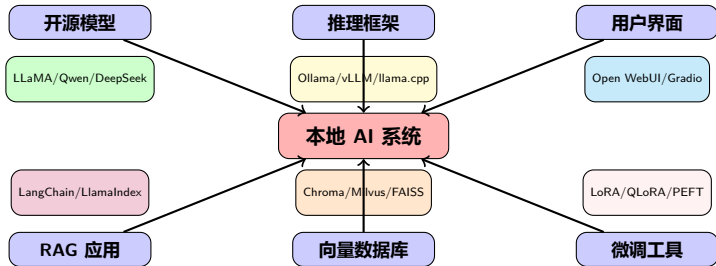
③ 敏感数据：离线部署

- 完全断网
- 专用服务器

推荐配置

办公场景：RTX 4090 (24GB) 运行 7B-13B 模型足够日常使用

附录 A：开源工具生态图



完整开源方案：无需付费，数据安全，可定制化

附录 B：国内用户访问方案

直接可用 (无需翻墙):

- ✓ DeepSeek - chat.deepseek.com
- ✓ 通义千问 - tongyi.aliyun.com
- ✓ Kimi - kimi.moonshot.cn
- ✓ 豆包 - doubao.com
- ✓ 文心一言 - yiyi.baidu.com

需要科学上网:

- ChatGPT / OpenAI API
- Claude / Anthropic API
- Gemini (部分地区可用)
- GitHub Copilot (正常可用)

API 中转服务推荐

云雾 AI (yunwu.ai) - 稳定的 API 中转:

- 支持 GPT/Claude/Gemini 等主流模型
- OpenAI 兼容格式, 接入方便
- 国内直连, 无需特殊网络

推荐方案

- 1 日常使用: DeepSeek (免费 + 中文最强)
- 2 编程任务: GitHub Copilot (直连)
- 3 长文本: Kimi (支持超长文档)
- 4 代码 IDE: Cursor (正常可用)
- 5 API 开发: 云雾 AI 中转服务

附录：云雾 AI (yunwu.ai) API 中转服务详解

什么是云雾 AI?

- 国内可直连的 API 中转服务
- 支持 GPT-5.2/Claude/Gemini 等主流模型
- **OpenAI 兼容格式**，几乎零成本迁移
- 按量付费，价格透明

核心功能入口:

- **在线聊天**: <https://yunwu.ai/console/playground>
- **获取 Token**: <https://yunwu.ai/console/token>
- **查看余额**: <https://yunwu.ai/console/topup>

使用提示

Python 调用示例

```
from openai import OpenAI

client = OpenAI(
    api_key="your-yunwu-token",
    base_url="https://yunwu.ai/v1"
)

response = client.chat.completions.create(
    model="gpt-5.2", # 或其他模型
    messages=[
        {"role": "user",
         "content": "你好!" }
    ]
)

print(response.choices[0].message.content)
```

使用提示

- 先在官网注册获取 API Token

附录：通过 NextChat 接入云雾 AI（推荐方案）

什么是 NextChat?

- GitHub 8.6 万 + Star 的开源项目
- 支持 Web/桌面/移动端全平台
- 可私有部署，数据本地存储
- 支持自定义 API 接口地址

配置云雾 AI 的方法：

- ① 访问 NextChat 设置页面
- ② 在“自定义接口”中设置：
 - API Key: 你的云雾 Token
 - Base URL: <https://yunwu.ai>
- ③ 选择需要的模型即可使用

NextChat 官方地址：

<https://nextchat.club>

自部署环境变量配置

```
# .env.local 文件
OPENAI_API_KEY=your-yunwu-token
BASE_URL=https://yunwu.ai

# 可选：设置访问密码
CODE=your-password

# 可选：自定义模型列表
CUSTOM_MODELS=+gpt-5.2,+claude-opus-4.5
```

快速体验

- ① 访问 <https://app.nextchat.club>
- ② 点击设置 → 自定义接口
- ③ 填入云雾 AI 的 Token 和地址
- ④ 开始聊天！

附录：更多软件接入云雾 API 的方法

1. Cursor IDE 配置

- 设置 → Models → OpenAI API Key
- 填入云雾 Token
- Override OpenAI Base URL:
`https://yunwu.ai/v1`

2. VS Code + Continue 扩展

```
// config.json
{
  "models": [{
    "title": "GPT-5.2 via Yunwu",
    "provider": "openai",
    "model": "gpt-5.2",
    "apiKey": "your-yunwu-token",
    "apiBase": "https://yunwu.ai/v1"
  }]
}
```

3. LangChain / Llamaindex

许达（未来院三室）

4. 命令行工具 (cURL)

```
curl https://yunwu.ai/v1/chat/completions \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "model": "gpt-5.2",
  "messages": [{"role": "user",
                  "content": "Hello!"}]
}'
```

通用接入原则

任何支持 **OpenAI API 格式** 的软件都可以接入：

- ① 将 API Key 替换为云雾 Token
- ② 将 Base URL 改为
`https://yunwu.ai/v1`
- ③ 选择云雾支持的模型名称

附录：免费资源大汇总

完全免费的 AI 服务：

- ★ DeepSeek - 网页版无限使用
- ★ Kimi - 超长文本处理
- ★ 通义千问 - 阿里全能助手
- ★ Google AI Studio - Gemini 免费额度
- ★ Colab + Gemini - 免费 GPU+AI

学生/教育免费：

- GitHub Education Pack
 - 免费 Copilot Pro
 - 需 edu 邮箱认证
- Azure for Students
 - \$100 免费额度
 - 可用于 OpenAI API

开源本地部署（永久免费）：

- Ollama - 一键运行本地模型
- LM Studio - 图形化界面
- 推荐模型：
 - Qwen2.5 (中文最佳)
 - Llama 3.2
 - DeepSeek-Coder
 - CodeLlama

零成本科研方案

DeepSeek(对话) + Kimi(长文) +
Copilot Free(代码) + Ollama(本地)
= 完整 AI 辅助科研工具链

附录：好 Prompt vs 坏 Prompt 对比

✗ 坏的 Prompt:

帮我写个Python程序

问题：太模糊，AI 不知道写什么

用最好的方法解决这个问题

问题：没有具体问题描述

这代码为什么不work？
[没有附上代码和错误信息]

问题：缺少必要上下文

✓ 好的 Prompt:

用Python实现一个LRU缓存类，要求：

1. 容量可配置
2. $O(1)$ 时间复杂度
3. 线程安全
4. 包含类型注解和docstring

我在用PyTorch训练ResNet时遇到：
`RuntimeError: CUDA out of memory`
环境：RTX 3080 10GB, batch=64
请分析原因并给出3种解决方案

Prompt 黄金法则

具体 + 上下文 + 期望格式 + 约束条件 = 高质量输出

附录：可直接复制的 System Prompt 模板

1. Python 专家模板：

```
You are a senior Python developer with
10+ years experience. Follow these rules:
1. Use Python 3.11+ features
2. Follow PEP8 and type hints
3. Write comprehensive docstrings
4. Prefer functional over OOP when simpler
5. Always consider edge cases
6. Suggest tests for critical functions
```

2. 论文写作助手：

```
You are an academic writing expert.
Guidelines:
1. Use formal, precise language
2. Avoid overclaiming (no "breakthrough")
3. Suggest citations as [Ref] placeholders
4. Follow Nature/Science style
5. Be objective and evidence-based
```

3. 代码审查专家：

```
You are a code reviewer. For each review:
1. Check for bugs and edge cases
2. Evaluate performance implications
3. Assess code readability
4. Suggest specific improvements
5. Rate severity: Critical/Major/Minor
```

4. 数据分析师：

```
You are a data scientist. When analyzing:
1. First understand the data structure
2. Check for missing values/outliers
3. Use appropriate statistical tests
4. Visualize results with matplotlib
5. Explain findings for non-experts
```

💡 提示：在 Claude/ChatGPT 的“Custom Instructions”中设置，或在 Cursor 的 Rules 中配置

附录：科研 workflows 实战案例

案例：用 AI 工具完成一篇论文的实验部分

任务：实现论文中的算法，跑实验，生成图表

Step 1: 理解算法

- 工具：Claude Opus 4.5 / Gemini
- 上传 PDF，让 AI 解释算法细节
- 生成伪代码和流程图

Step 2: 代码实现

- 工具：Cursor + Claude
- 用 Agent 模式生成代码框架
- 逐步实现各个模块

Step 3: 调试修复

Step 4: 运行实验

- 工具：Claude Code (终端)
- 用 AI 管理实验配置
- 自动记录结果

Step 5: 分析结果

- 工具：ChatGPT Code Interpreter
- 上传 CSV，自动统计分析
- 生成对比表格

Step 6: 生成图表

- 工具：Cursor + Matplotlib

附录：不同任务的最佳工具组合

任务类型	推荐工具组合	说明
日常编程	Cursor + Claude Opus	最强代码体验
预算有限编程	VS Code + Copilot Free + DeepSeek	零成本方案
论文阅读总结	Gemini 3 Pro / Kimi	超长上下文
中文论文写作	DeepSeek + Claude 润色	中文理解 + 英文润色
数学推导	DeepSeek V3.2 / GPT-5.1 Pro	推理能力强
数据分析	ChatGPT Code Interpreter	直接执行代码
图表生成	Cursor + Matplotlib	快速迭代
代码审查	Claude Opus 4.5	理解代码最深
Debug 调试	Copilot Chat /fix	快速定位问题
学习新技术	任意模型 + 官方文档	配合官方资料

核心原则

没有万能工具，根据任务特点选择最合适的组合。熟练掌握 2-3 个工具比浅尝辄止更有效。

附录：价格速查表

产品	免费版	个人版	团队/企业版
ChatGPT Plus	有限使用	\$20/月	\$25-30/用户/月
Claude Pro/Max	有限使用	\$20-200/月	\$30/用户/月起
Gemini Advanced	免费额度	\$20/月	企业定制
GitHub Copilot	有限功能	\$10-39/月	\$19-39/用户/月
Cursor	有限使用	\$20-200/月	\$40/用户/月
Windsurf	免费版可用	定制	企业定制
DeepSeek	完全免费	API 按量	API 按量

省钱技巧

- 学生/教师可申请 GitHub Education Pack (免费 Copilot)
- 开源项目维护者可申请免费额度
- 合理使用缓存和批处理 API 降低成本
- DeepSeek 提供极具竞争力的免费服务

附录：快捷键速查

GitHub Copilot (VS Code):

- Tab - 接受建议
- Esc - 拒绝建议
- Alt+] - 下一个建议
- Alt+[- 上一个建议
- Ctrl+Enter - 打开建议面板
- Ctrl+I - 打开 Copilot Chat

Cursor:

- Tab - 接受补全
- Cmd/Ctrl+K - 行内编辑
- Cmd/Ctrl+L - 打开 Chat
- Cmd/Ctrl+I - Composer

Copilot Chat 命令:

- /explain - 解释代码
- /fix - 修复问题
- /tests - 生成测试
- /doc - 生成文档
- /optimize - 优化代码
- @workspace - 项目上下文
- @vscode - 编辑器操作

Claude Code:

- 直接在终端输入 `claude`
- 支持管道输入
- 可以执行 shell 命令

不同任务的最佳模型选择 (2026.01 版)

- **长文本阅读与润色**: Gemini 3 Pro (2M context), Claude Opus 4.5 (200k)
 - 理由: 超长上下文窗口, 能一次性读完整本书或多篇论文, 记忆力强。
- **图形修改与多模态**: Gemini 3 Pro
 - 理由: 原生多模态能力最强, 能精准理解图表细节并提出修改建议。
- **数学推理与理论推导**: DeepSeek V3.2 (中文最强), GPT-5.2 Pro
 - 理由: DeepSeek 在数学和逻辑推理上已达到 GPT-5.2 同级水平, 且对中文数学术语支持最好。
- **代码实现与实验**: Claude Opus 4.5, GPT-5.2
 - 理由: Claude 4.5 是目前公认的代码之王, GPT-5.2 紧随其后。

附录：科研全流程 Prompt 指南 (2/6) - 选题与文献

1. 选题与头脑风暴 (Brainstorming)

Prompt 示例 (加入探索/联想关键词)

"I am researching [field/topic], and the current pain point is [specific problem]. Please explore potential research directions by associating with the latest trends in [conference/journal]. Discover 3-5 innovative ideas. For each: 1. Core novelty; 2. Feasibility analysis; 3. Potential challenges."

2. 文献调研与总结 (Literature Review)

Prompt 示例 (推荐模型: Gemini 3 Pro / Claude Opus 4.5)

"Please read the 5 uploaded PDF papers. Summarize them for a beginner. Requirements: 1. Extract the core contribution of each paper; 2. Compare their methodological similarities and differences (create a comparison table); 3. Identify and discover the common limitations, and explore possible future improvements."

3. 数学公式推导 (Mathematical Derivation)

Prompt 示例 (推荐模型: DeepSeek V3.2, GPT-5.2 Pro)

"I have defined a loss function $L = \dots$. Please derive the gradient $\nabla_x L$ with respect to variable x . Show the derivation step by step, and check for potential gradient vanishing or explosion risks."

4. 理论证明检查 (Proof Checking)

Prompt 示例

"Here is my proof draft for Theorem X: [paste proof]. Act as a rigorous reviewer. Check for logical gaps, especially whether the assumptions are sufficient and whether the derivation steps are rigorous. If there are errors, point out the exact location and provide correction suggestions."

5. 算法实现 (Implementation)

Prompt 示例 (推荐模型: Claude Opus 4.5)

```
"Please implement Algorithm 1 from the appendix using PyTorch. Requirements: 1. PEP8 compliant code style; 2. Include detailed type annotations and docstrings; 3. Optimize for GPU parallel computation; 4. Provide a simple dummy data test case to verify dimension correctness."
```

6. 实验报错调试 (Debugging)

Prompt 示例

```
"My code encountered the following error: [paste error message]. Here is the relevant code snippet: [paste code]. Please analyze the root cause and provide the fixed code. Explain why this error occurred and how to avoid it in the future."
```

附录：科研全流程 Prompt 指南 (5/6) - 数据与绘图

7. 数据分析 (Data Analysis)

Prompt 示例 (推荐模型: GPT-5.2 Code Interpreter)

"I have uploaded the experimental results CSV file. Please analyze: 1. The impact of different hyperparameters on model performance; 2. Whether there is a statistically significant difference (perform t-test); 3. Identify and discover the Pareto-optimal configurations."

8. 科学绘图与修改 (Visualization)

Prompt 示例 (推荐模型: Gemini 3 Pro)

"Please use Python Matplotlib to create a line chart comparing Method A and Method B. Use academic-style colors (e.g., Science journal style), Times New Roman font, size 12." "(Upload image) The legend in this figure blocks the data curve. Please move the legend to the upper left corner, change the blue curve to dashed line, and make the red curve thicker."

9. 论文写作 (Writing)

Prompt 示例 (推荐模型: Claude Opus 4.5)

"Here are my experimental results and method description. Please write the Introduction section. Requirements: 1. Use inverted pyramid structure; 2. Emphasize the novelty of our method; 3. Professional and objective language, avoid overclaiming; 4. Cite relevant classic literature (use [Ref] as placeholder)."

10. 审稿回复 (Rebuttal)

Prompt 示例

"The reviewer raised this question: [paste question]. We have actually discussed this in the experimental section (see Section 4.2). Please help me draft a polite yet strong response. Requirements: 1. Thank the reviewer for the suggestion; 2. Point to the corresponding content in our paper; 3. Add a new explanation to further clarify."

何时使用“探索/联想/发现”关键词？

- **Explore**：当需要模型发散思考、提出多种可能性时
- **Associate / Connect**：当希望模型找到不同概念之间的联系时
- **Discover / Identify**：当希望模型从数据或文本中挖掘隐藏的规律时
- **Brainstorm**：当需要创意和头脑风暴时

高级 Prompt 示例 (带探索关键词)

```
"Please explore unconventional approaches to solve [problem]. Associate techniques from [Field A] with methods in [Field B]. Discover potential synergies. Think step-by-step and brainstorm at least 5 creative solutions, even if some seem unconventional."
```


第七部分：总结与展望

关键点回顾

大模型选择:

- ① GPT-5 系列 - 全能选手
- ② Claude 4.5 - 代码之王
- ③ Gemini 3 - 多模态强手
- ④ DeepSeek V3 - 性价比首选 (开源)

开源工具选择:

- ① Ollama + 本地模型 - 隐私保护
- ② Continue.dev - 开源 Copilot
- ③ Zotero - 文献管理
- ④ LangChain + Chroma - 知识库

实用技能清单:

- ✓ 掌握至少一个 AI 编程工具
- ✓ 学会基础 Prompt 工程
- ✓ 了解不同模型特点
- ✓ 能在科研中合理应用 AI
- ✓ 知道如何控制成本
- ✓ 了解数据隐私保护

核心原则

AI 是**增强**而非**替代**

保持批判性思维, 验证 AI 输出

谢谢!

问题与讨论

-  <https://github.com/copilot>
-  <https://cursor.com>
-  <https://claude.ai>
-  <https://chat.deepseek.com>

演示文稿更新日期：2026 年 1 月 7 日

数据来源：Scale AI HLE Leaderboard, SWE-bench, OpenAI, Anthropic, Google AI Blog, Cursor Changelog